

TECHNOLOGY OF COMPILERS LAB REPORT



IMPLEMENTATION OF LEXICAL ANALYZER

Name: IGBO KENNETH CHIAGOZIE

ID: 5112100502

Class: COMPUTER SCIENCE 3RD YEAR 2023/2024 SESSION

Teacher: NIU, DE JIAO

Date: May 26, 2024

PURPOSE AND CONTENT

1. Implement and understand the aspect and implementation of lexical analyzers.

CONTENT

1. Source codes and their respective outputs.
2. Explanations and illustrations of ideas used.

LEXICAL ANALYZERS

Lexical analyzers are also known as lexical scanners or lexical tokenizers. They are the initial step in the analysis of source code which involves converting a sequence of characters into a sequence of tokens.

Tokens are the basic building blocks of language syntax such as keywords, literals, identifiers, operators and punctuation symbols.

Lexical Analysis Process

1. Reading Input:

This involves the reading of the raw source code character by character.

2. Character Grouping:

This involves the grouping of characters accordingly into lexemes depending on the rules for the particular language's grammar.

3. Token Generation:

This involves the conversion of each lexeme to a token. The token is usually a pair consisting of a token type and an optional number which denotes position in the corresponding symbol table.

TOKEN TYPES

For this lab experiment, I used the following types of tokens:

1. Integers:

This denotes all integers. The token here contains 2 values, the token type and the position on the constant table.

Editor:	Output Tokens:	Constant Table:				
22	<INTEGER, 1>	<table border="1"><thead><tr><th>Pos</th><th>Constant Values</th></tr></thead><tbody><tr><td>1</td><td>25</td></tr></tbody></table>	Pos	Constant Values	1	25
Pos	Constant Values					
1	25					

OUTPUT FOR 22

2. Keywords:

This denotes all keywords included in the keyword list. It only contains one output value.

Editor:	Output Tokens:
int	<'int'>

Output for int

```
keyword_list = ["int", "main", "not", "or", "and", "for", "while", "else", "if", "True", "False", "None", "NULL",
               "do", "class", "float", "double", "bool", "cout", "break", "continue", "def", "endl", "print", "cin",
               "void", "using", "namespace", "std", "include", "iostream"]
```

3. Identifier:

This denotes variables names used in codes. It contains 2 values, the first is ID denoting the token type and the second is the position in the variable symbol table.

Editor:	Output Tokens:	Variable / Identifier Table:
var1	<ID, 1>	Pos Variable Names 1 var1

Output for var1

4. Symbol:

These are the symbols in the symbol list. Its token contains one value which is the symbol.

Editor:	Output Tokens:
;	<'!';!>

Output for ;

```
symbol_list = [" ", ":", ":", ":", "\"", "/", "+", "=", "_", "!", "@", "#", "&", "^", ".", "{", "}", "[", "]", "!",
               "(", ")", "\\ ", "<", ">", "?", "~", "*"]
```

IMPLEMENTATION

For the implementation, python programming language was used. The major sections, functions and classes are as follows:

1. Token Class:

This is the class that denotes the Token.

```
class Token:
    def __init__(self, type, value, symbol_table=None):
        self.type = type
        self.value = value
        self.table_pos = 0
        self.symbol_Table = symbol_table

    def __str__(self):
        classified = self.classify_value()
        if classified is not None:
            return f'<{repr(self.value)}>'
        if self.type == INTEGER:
            self.table_pos = self.symbol_Table.find_in_constant_table(self.value)
            if self.table_pos == -1:
                self.table_pos = self.symbol_Table.add_to_constant_table(self.value)
        else:
            self.table_pos = self.symbol_Table.find_in_variable_table(self.value)
            if self.table_pos == -1:
                self.table_pos = self.symbol_Table.add_to_variable_table(self.value)
        return f'<{self.type}, {repr(self.table_pos)}>'

    def __repr__(self):
        return self.__str__()

1 usage
    def classify_value(self):
        if self.value in keyword_list:
            return "keyword"
        elif self.value in symbol_list:
            return "symbol"
        return None
```

2. Symbol Tables Class:

This is a class that denotes the variable and constant tables.

```
class SymbolTables:
    def __init__(self):
        self.variable_table = []
        self.constant_table = []
        self.variable_table_length = 0
        self.constant_table_length = 0

    2 usages (2 dynamic)
    def add_to_variable_table(self, value):
        self.variable_table.append([self.variable_table_length + 1, value])
        self.variable_table_length += 1
        return self.variable_table_length

    2 usages (2 dynamic)
    def add_to_constant_table(self, value):
        self.constant_table.append([self.constant_table_length + 1, value])
        self.constant_table_length += 1
        return self.constant_table_length

    2 usages (2 dynamic)
    def find_in_constant_table(self, value):
        for i in self.constant_table:
            if i[1] == value:
                return i[0]
        return -1

    2 usages (2 dynamic)
    def find_in_variable_table(self, value):
        for i in self.variable_table:
            if i[1] == value:
                return i[0]
        return -1
```

3. Lexer/Lexeme Class:

This denotes the lexeme or lexer class and it is used to classify the lexemes and read the source file character by character.

```
class Lexer:
    def __init__(self, text):
        self.text = text
        self.pos = 0
        self.current_char = self.text[self.pos]
        self.flag = False

    1 usage
    def error(self, type=1):
        if type == 1:
            raise Exception(f'Invalid character {self.current_char}')

    4 usages
    def advance(self):
        self.pos += 1
        if self.pos > len(self.text) - 1:
            self.current_char = None
        else:
            self.current_char = self.text[self.pos]

    def retract(self):
        self.pos -= 1
        if self.pos > len(self.text) - 1:
            self.current_char = None
        else:
            self.current_char = self.text[self.pos]

    1 usage
    def skip_whitespace(self):
        cond = self.current_char in symbol_list
        while self.current_char is not None and (self.current_char.isspace() or self.current_char == "\n") and not cond:
            self.advance()
            cond = self.current_char in symbol_list

    def integer(self):
        result = ''
        while self.current_char is not None and self.current_char.isdigit():
            result += self.current_char
            self.advance()
        if self.current_char is not None and self.current_char.isalpha():
            self.current_char = result + self.current_char
            # self.error()
            return "error"
        return int(result)

    1 usage
    def string(self):
        result = ''
        while self.current_char is not None and self.current_char.isalnum():
            result += self.current_char
            self.advance()
        if self.current_char in symbol_list:
            self.flag = True
        return result

    2 usages
    def get_next_token(self, symbol_Table):
        while self.current_char is not None:
            if self.current_char.isspace() or self.current_char == "\n":
                self.skip_whitespace()
                continue

            if self.current_char.isdigit():
                temp_ret = self.integer()
                if temp_ret != "error":
                    return Token(INTEGER, temp_ret, symbol_Table)

            if self.current_char.isalpha():
                return Token(IDENTIFIER, self.string(), symbol_Table)

            if self.current_char in symbol_list:
                temp_ret = self.current_char
                self.advance()
                return Token(SYMBOL, temp_ret)

            self.error()
            break

        return Token(EOF, value=None)
```

4. detection_thread function:

This is the function which performs the detection of the input in the GUI editor text box. It runs on a new thread. It ensures that the compilation and analysis of the input is only done when there is a change registered. It also allows for pausing and continuation of detection thread.

```
def detection_thread():
    global output_textbox, input_textbox
    textbox_string = ""
    while True:
        check_queue()
        if textbox_string != input_textbox.get(index1: 1.0, index2: "end-1c"):
            symbol_Table = SymbolTables()
            textbox_string = input_textbox.get(index1: 1.0, index2: "end-1c")
            output_textbox.config(state="normal")
            output_textbox.delete(index1: 1.0, index2: "end")
            output_textbox.insert(index: 1.0, chars: f"{analyze(textbox_string, symbol_Table)}")
            output_textbox.config(state="disabled")
            variable_textbox.config(state="normal")
            variable_textbox.delete(index1: 1.0, index2: "end")
            variable_textbox.insert(index: 1.0, chars: f"{get_variable_table_text(symbol_Table)}")
            variable_textbox.config(state="disabled")
            constant_textbox.config(state="normal")
            constant_textbox.delete(index1: 1.0, index2: "end")
            constant_textbox.insert(index: 1.0, chars: f"{get_constant_table_text(symbol_Table)}")
            constant_textbox.config(state="disabled")
```

5. analyze function:

This is the function used to analyze the input token and is called by the detection thread function. It gets the tokens, and uses them to update the symbol and constant table accordingly. Afterwards it returns the analyzed text which are tokens.

```
def analyze(text, symbol_Table):
    analyzed_text = ""
    lexer = Lexer(text + "\n")
    try:
        token = lexer.get_next_token(symbol_Table)
    except Exception as e:
        analyzed_text += f"Error at {str(e)[18:]} \n"
        token = Token(EOF, value: "")

    while token.type != EOF:
        analyzed_text += str(token) + "\n"
        try:
            token = lexer.get_next_token(symbol_Table)
        except Exception as e:
            analyzed_text += f"Error at {str(e)[18:]} \n"
            break
    return analyzed_text
```

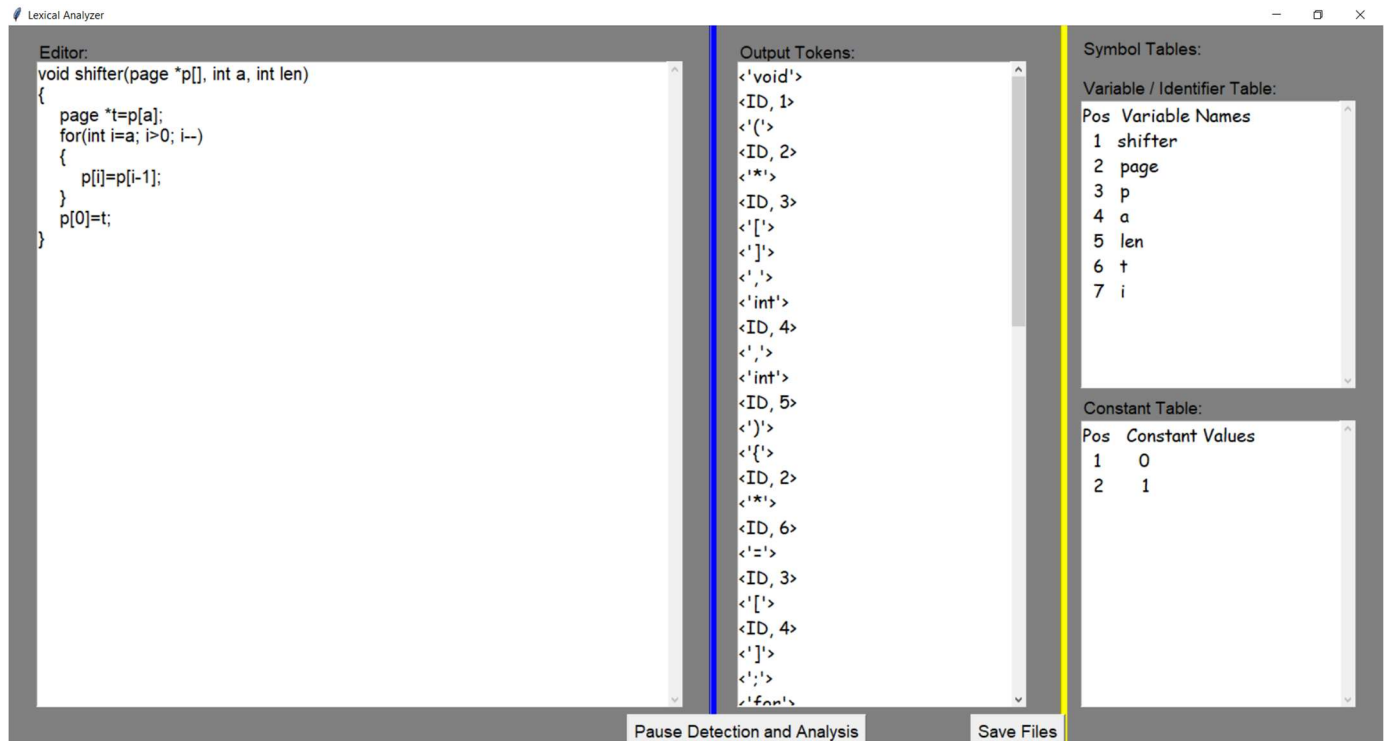

6. Main function:

This is the main function which indicates the start of the code. It creates the file path, initializes the threading event and instruction queue. Afterwards it creates and initializes the GUI and starts the detection thread.

```
if "__main__" == __name__:  
    createPath()  
    global event, instruction_queue, paused  
    event = threading.Event()  
    instruction_queue = queue.Queue()  
    paused = False  
    root = tk.Tk()  
    initGUI(root)  
    detection()  
    root.mainloop()
```

TESTING AND OUTPUT

For the testing and output, the picture is displayed below:



While the tokens, symbol and output table can be seen in the picture, I will also include the files which display the input code, output token, variable table and constant table individually.

For a simpler testing of the lexical analyzer, the picture can be shown below:

Lexical Analyzer

Editor:

```
int length = 30;  
int breadth = 60;  
int width = 90;  
int area = length * breadth;
```

Output Tokens:

```
<'int'>  
<ID, 1>  
<'='>  
<INTEGER, 1>  
<';'>  
<'int'>  
<ID, 2>  
<'='>  
<INTEGER, 2>  
<';'>  
<'int'>  
<ID, 3>  
<'='>  
<INTEGER, 3>  
<';'>  
<'int'>  
<ID, 4>  
<'='>  
<ID, 1>  
<'*>  
<ID, 2>  
<';'>
```

Symbol Tables:

Variable / Identifier Table:

Pos	Variable Names
1	length
2	breadth
3	width
4	area

Constant Table:

Pos	Constant Values
1	30
2	60
3	90

Pause Detection and Analysis

Save Files

CONCLUSION

From this lab report, it can be seen that the lexical analyzer is one of the most important aspects of the compilation process. This is because it denotes and recognizes lexemes which are then classified into tokens.